
CSE 312 — İşletim Sistemleri

Bölüm 2.3 & 2.6

Synchronization & Deadlocks

Ders: CSE 312 — Operating Systems (Spring 2026)

Kaynak: Tanenbaum, Modern Operating Systems, 5th Ed. — Chapter 2.3, 2.5, 2.6

Öğretim Üyesi: Prof. Dr. Yakup Genç — Gebze Teknik Üniversitesi

Konu: Senkronizasyon, Klasik Problemler, Deadlock — Tespit, Kaçınma, Önleme

. İindekiler

1. Zamanlayıcılar ve Senkronizasyon Temelleri

- 1.1 Jiffies — 32-bit Tařma ve 64-bit Eriřim Sorunu
- 1.2 Potent and Evil Magic: Spin-Lock Çözümü
- 1.3 Time of Day — xtime ve adjtimex()
- 1.4 Kernel Timers: Dynamic Timer ve Delay Loop
- 1.5 Dynamic Timer: Parametre, İptal ve Senkronizasyon
- 1.6 Delay Functions: Busy-Wait Kullanımı
- 1.7 Sistem Çağrılar: time(), gettimeofday(), alarm()
- 1.8 Watchdog Timer ve Gömülü Sistemler

2. Klasik Senkronizasyon Problemleri

- 2.1 Neden Klasik Problemler?
- 2.2 Dining Philosophers — Problem Tanımı
- 2.3 Düşünmek ve Yemek Yemek: Senaryo
- 2.4 Hatalı İlk Çözüm — Deadlock
- 2.5 Onarım Giriřimi — Deadlock Değil, Starvation
- 2.6 Rastgeleleřtirme (Randomization) — Knuth'tan Alıntı
- 2.7 Basit Mutex Çözümü ve Sorunu
- 2.8 Doğru ve Adil Çözüm — İspat Taslağı
- 2.9 Readers and Writers Problemi
- 2.10 Dersler ve Genel İlkeler

3. Deadlock (Kilitlenme)

- 3.1 Deadlock Nedir? Alice ve Bob Senaryosu
- 3.2 Tarihe — OS'ten Uygulama Sorununa
- 3.3 Kaynaklar: Fiziksel, Soyut, Preemptable
- 3.4 Kaynak Kullanım Dizisi
- 3.5 Coffman Kořulları (1971) — 4 Gerekli Kořul
- 3.6 Kaynak Atama Grafiğı ile Modelleme
- 3.7 Deadlock'tan Kaınma — Graf Analizi
- 3.8 Genel Stratejiler

4. Deadlock ile Bařa Çıkma — 4 Strateji

-
- 4.1 Görmezden Gelme — Ostrich Algoritması
 - 4.2 Tespit ve Kurtarma — Tek Kaynak Türü
 - 4.3 Tespit ve Kurtarma — Çoklu Kaynak Türü
 - 4.4 Deadlock Örneği Adım Adım
 - 4.5 Kurtarma Yöntemleri: Preemption, Rollback, Kill
 - 4.6 Deadlock Kaçınma — Proses Trajektör Analizi
 - 4.7 Güvenli ve Güvensiz Durumlar
 - 4.8 Banker's Algoritması (Dijkstra, 1965)
 - 4.9 Banker's Çoklu Kaynak İçin
 - 4.10 Banker's Neden Kullanılmıyor?
 - 4.11 Deadlock Önleme — 4 Koşula Saldırı
 - 4.12 İki Aşamalı Kilitleme (Two-Phase Locking)
 - 4.13 Linux'ta Deadlock — Fork Bomb

5. Sınav Soruları ve Çözümleri

6. Kitap Sonu Soruları ve Çözümleri

1. Zamanlayıcılar ve Senkronizasyon Temelleri

Linux kernel timer altyapısı — Senkronizasyon ihtiyacının ilk tezahürleri

1.1 Jiffies — 32-bit Taşma ve 64-bit Erişim Sorunu

Linux çekirdeği, sistem başlatılmasından bu yana geçen süreyi jiffies adı verilen bir sayaçla izler. Her donanım saat kesmesinde (timer interrupt) bu değişken bir artırılır. Sistemin HZ parametresi (genellikle 250 ya da 1000 Hz) saniyede kaç kesme üretileceğini belirler. Dolayısıyla jiffies değeri, sistem başlangıcından bu yana geçen yaklaşık kesme sayısına eşittir.

⚠ 32-bit Taşma Problemi

1000 Hz'de çalışan 32-bit bir jiffies sayacı yalnızca yaklaşık 49.7 günde sıfıra döner (wrap-around). Uzun süreli çalışan sunucularda bu durum, zaman karşılaştırmalarında yanlış sonuçlara ve ciddi hatalara yol açar.

Çözüm olarak 64-bit jiffies_64 sayacı tanımlanmıştır. 64-bit bir değer 1000 Hz'de yaklaşık 585 milyon yıl önce taşardı — bu sorun artık yok. Ancak yeni bir sorun ortaya çıkar: 32-bit bir makinede 64-bit değere atomik erişim mümkün değildir.

1.2 Potent and Evil Magic: Spin-Lock ve Linker Çözümü

64-bit bir değer 32-bit makinede iki ayrı 32-bit okuma işlemi gerektirmektedir. Bu iki okuma arasında bir saat kesmesi gelirse, sayacın üst 32 biti henüz güncellenmemiş iken alt 32 bit okunmuş ya da tam tersi olabilir. Sonuçta tutarsız (saçma) bir değer elde edilir. Bu, klasik bir race condition (yarış koşulu) örneğidir.

i Çözüm: Spin-Lock + Linker Magic

Kernel, jiffies_64 değerine erişimi bir spin-lock ile korur. Uygulama kodu doğrudan jiffies_64'e erişmez; bunun yerine get_jiffies_64() çağrısını kullanır. Ayrıca bağlayıcı (linker) bir hile uygular: "jiffies" sembolü, jiffies_64'ün düşük anlamlı 32 bitine işaret eder. Böylece her saat kesmesinde yalnızca jiffies_64 bir kez artırılır; hem "jiffies" hem de "jiffies_64" güncel kalır. Zarif bir çözüm değil — ama işe yarıyor.

1.3 Time of Day — xtime ve adjtimex()

Sistemin gerçek duvar saatini (wall-clock time) tutmak için çekirdek xtime adlı bir struct timespec kullanır. Bu yapı her saat kesmesinde güncellenir ve erişimi yine bir spin-lock ile korunur. adjtimex() sistem çağrısı, kristal osilatörlerin zaman içinde yaşadığı küçük sapmalar (drift) nedeniyle görünür saat hızını ince ayarlamak için kullanılır. NTP (Network Time Protocol) bu mekanizmanın üzerine inşa edilmiştir.

1.4 Kernel Timers: Dynamic Timer ve Delay Loop

Timer Türü	Mekanizma	Gecikme Boyutu	Kullanım Durumu
Dynamic Timer	Gelecekte bir zamana callback fonksiyon kaydedilir	Milisaniye ve üzeri	Ağ zaman aşırımları, disk I/O timeout, zamanlı iş kuyruğu
Delay Loop (Spin)	CPU döngüde bekletilir — busy-wait	Mikrosaniye / nanosaniye	Dynamic timer kurma yükünün çok kısa gecikmeler için kabul edilemez olduğu durumlar

1.5 Dynamic Timer: Parametre, İptal ve Senkronizasyon

Bir dynamic timer kurulurken üç bilgi verilir: (1) ne kadar süre sonra ateşleneceği, (2) çağrılacak callback fonksiyonu, (3) bu fonksiyona geçirilecek parametre. Parametre, aynı callback şablonunu kullanan farklı örnekleri birbirinden ayırt etmek için kullanılır.

Örnek: 4 Ethernet Kartı

Bir sistemde 4 Ethernet kartı aynı timeout fonksiyonunu kullanıyorsa, her kart kendi veri yapısının adresini parametre olarak geçirir. Fonksiyon çağrıldığında hangi kartla ilgili olduğunu bu parametre sayesinde anlar.

⚠ Çok İşlemcili Sistemlerde Timer İptali

Bir timer iptal edilmek istendiğinde, o timer'ın callback fonksiyonu başka bir CPU'da tam o anda çalışıyor olabilir. Linux bu durumu özel kilitleme mekanizmalarıyla ele alır. Bu "ince senkronizasyon dansı", çok işlemcili sistemlerde doğru timer iptali için zorunludur.

1.6 Delay Functions: Busy-Wait Kullanımı

Delay fonksiyonları CPU'yu yoğun bir döngüye sokarak (tight spin loop) bekletir. Bu süre boyunca o CPU'da başka hiçbir process çalıştırılmaz — yalnızca kesmeler tetiklenebilir. Bu nedenle delay fonksiyonları yalnızca nanosaniye ya da mikrosaniye mertebesindeki çok kısa gecikmeler için tercih edilir; daha uzun bekleme için dynamic timer veya `schedule_timeout()` kullanılmalıdır.

1.7 Sistem Çağrıları

Çağrı	İşlev	Açıklama
<code>time()</code>	Zaman okuma	1 Ocak 1970 epoch'tan bu yana geçen saniyeyi döner
<code>gettimeofday()</code>	Hassas zaman okuma	Mikrosaniye hassasiyetinde günün zamanını döner (struct timeval)
<code>adjtimex()</code>	Saat hızı ayarı	Kristal sapmasını düzeltmek için görünür saat hızını ince ayarlar (NTP kullanır)
<code>setitimer()</code>	Aralıklı timer	Belirtilen süre sonra ve sonrasında periyodik SIGALRM sinyali üretir
<code>alarm()</code>	Tek seferlik uyarı	N saniye sonra SIGALRM gönderir; 0 ile iptal edilir

1.8 Watchdog Timer ve Gömülü Sistemler

Watchdog (bekçi köpeği) timer, bir sistemin düzgün çalışıp çalışmadığını denetlemek için kullanılan bir donanım/yazılım mekanizmasıdır. Çalışma prensibi şöyledir: sistem yazılımı periyodik olarak watchdog sayacını sıfırlamak zorundadır. Eğer sayaç belirlenen süre dolmadan sıfırlanmazsa — yani sistem kilitlenmiş veya çökmüş demektir — watchdog donanımı sistemi otomatik olarak yeniden başlatır.

i COP Timer — Computer Operating Properly

Watchdog timer özellikle gömülü (embedded) ve uzak otomatik sistemlerde kritik önem taşır; çünkü bu sistemlerde bir kullanıcı gidip cihazı elle yeniden başlatamaz. Örneğin bir trafik kontrol sistemi, sanayi robotu veya uydu.

Gömülü Sistem: Daha büyük bir mekanik ya da elektriksel sistemin içine yerleştirilmiş, kendine özgü bir işlevi olan ve çoğunlukla gerçek zamanlı kısıtlamalar taşıyan bilgisayar sistemidir. ADSL modem, araç kontrol birimi, tıbbi cihaz, sanayi PLC gibi örnekler verilebilir.

Mikrodenetleyici (Microcontroller): İşlemci çekirdeği, bellek ve programlanabilir giriş/çıkış birimlerini tek bir entegre devre üzerinde barındıran küçük bilgisayardır. Intel 8742, 12 MHz'de çalışan 8-bit CPU, 128 bayt RAM ve 2 KB EPROM'dan oluşur.

i Mars Pathfinder ve Watchdog

NASA'nın Mars Pathfinder uzay aracında bir Priority Inversion hatası, yüksek öncelikli thread'in zamanında çalışamamasına yol açtı. Watchdog timer bu thread'in çalışmadığını fark edince sistemi sürekli sıfırladı. Sorun, uzaktan priority inheritance mekanizması aktive edilerek çözüldü — watchdog'un önemi bu vakada açıkça görülmüştür.

2. Klasik Senkronizasyon Problemleri

Dining Philosophers · Readers and Writers — İlkeler ve Çözümler

2.1 Neden Klasik Problemler?

Klasik senkronizasyon problemleri, mutex, semaphore ve monitör gibi senkronizasyon ilkellerin nasıl kullanılacağını göstermek amacıyla icat edilmiştir. Gerçek dünya senaryolarını birebir yansıtmak için tasarlanmamışlardır; bununla birlikte içerdikleri prensiplerin doğrudan gerçek çözümlerde kullanıldığı görülür.

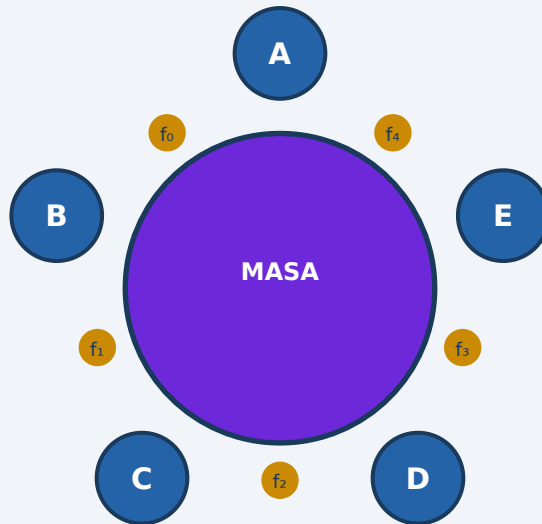
i Klasik Problemlerin Değeri

Bu problemler senkronizasyon hatalarının temel kalıplarını — deadlock, starvation, race condition — soyutlanmış ve anlaşılır ortamlarda sergilediği için hem öğretim hem de algoritma tasarımı açısından son derece değerlidir.

2.2 Dining Philosophers — Problem Tanımı

Dijkstra tarafından 1965 yılında formüle edilen bu problem, paylaşılan kaynaklar için rekabet eden proseslerin senkronizasyonunu modellemektedir. Senaryo şöyledir: beş filozof yuvarlak bir masada oturmaktadır. Her filozofun önünde bir tabak makarna vardır. Her iki tabak arasında birer çatal bulunur — toplamda 5 çatal. Bir filozofun yemek yiyebilmesi için iki çatala birden (solundaki ve sağındaki) ihtiyacı vardır.

Dining Philosophers — 5 Filozof, 5 Çatal



Şekil 1: 5 filozof, 5 çatal — normal durum. Mavi daireler filozoflar, sarı noktalar serbest çatallardır.

2.3 Düşünmek ve Yemek Yemek

Filozoflar ömürlerini düşünmek ve yemek yemek arasında geçirir. Düşünürken hiçbir kaynağa ihtiyaç duymazlar ve sonsuza dek düşünebilirler. Yemek yemek için önce sol çatalı, sonra sağ çatalı almaları gerekir. Yedikten sonra her iki çatalı bırakırlar ve tekrar düşünmeye başlarlar.

2.4 Hatalı İlk Çözüm — Deadlock

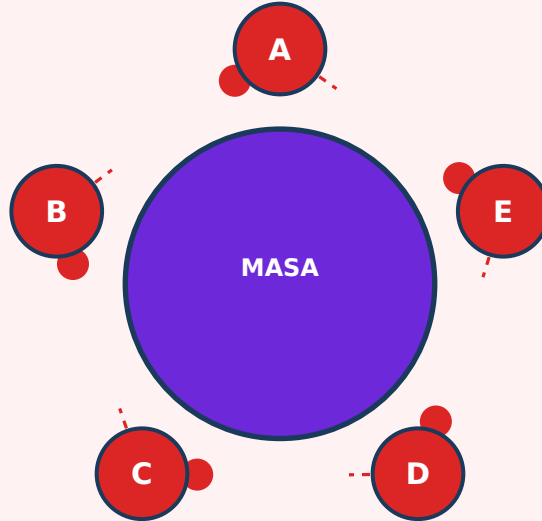
Akla ilk gelen çözüm her filozofu şu şekilde kodlamaktır:

```
void philosopher(int i) {
    while (TRUE) {
        think();           // Düşün
        take_fork(i);      // Sol çatalı al (i numaralı çatal)
        take_fork((i+1) % N); // Sağ çatalı al
        eat();             // Ye
        put_fork(i);       // Sol çatalı bırak
        put_fork((i+1) % N); // Sağ çatalı bırak
    }
}
```

⚠ Bu Çözüm Hatalı — Deadlock Kaçınılmaz

Tüm filozoflar tam aynı anda sol çatallarını alırlarsa: her biri bir çatal tutmakta ve sağdaki çatalı beklemektedir. Sağdaki çatal ise komşunun sol çatalıdır — o da tutulmuş durumdadır. Hiçbiri ilerleyemez. Circular wait koşulu oluşmuştur: DEADLOCK.

DEADLOCK — Herkes 1 Çatal Tutarken Sağdakini Bekliyor



Şekil 2: Deadlock durumu — her filozof 1 çatal tutarken sağdakini sonsuza dek bekliyor (kesik oklar beklemeyi gösterir).

2.5 Onarım Girişimi — Starvation

Sağ çatal müsait değilse önce kontrol edip sonra al denilebilir. Ancak kontrol ve alma işleminin kritik bölge içinde atomik yapılması gerekir. Çatal müsait değilse proses uyku moduna geçer ve tekrar dener. Bu yaklaşım deadlock'u önler gibi görünse de başka bir soruna yol açar:

⚠ Starvation (Açlık) Tehlikesi

Tüm filozoflar aynı anda sol çatalı alır, sağ bulamazlar, aynı süre uyurlar, aynı anda uyanırlar ve yeniden aynı anda sol çatalı alırlar. Herkes çalışır ama hiçbir ilerleme kaydedemez — bu duruma livelock veya starvation denir.

2.6 Rastgeleleştirme (Randomization)

Filozoflar ikinci çatalı alamazlarsa rastgele bir süre uyurlarsa, büyük olasılıkla bir dahaki denemede başarılı olurlar. Çünkü farklı rastgele bekleme süreleri senkronizasyonu bozar. Bu yaklaşım çoğu zaman işe yarar.

⚠ Knuth'tan Uyarı — Rastgeleleştirme Yeterli Değil

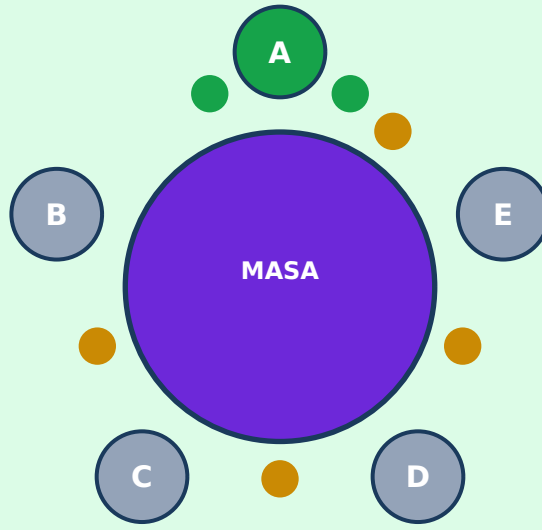
"Olasılıksal çözümleri kullanırken olasılık teorisine büyük güven duymak gerekir — bu yöntemler yalnızca ortalamada verimlidir, en kötü durumları korkunçtur." Uçak trafik kontrolü gibi gerçek zamanlı sistemlerde "genellikle çalışır" kabul edilemez; kesin garantiler gereklidir.

2.7 Basit Mutex Çözümü ve Sorunu

Çatal almadan önce global bir mutex kilitlenerek, iki çatal alma işleminin atomik biçimde tamamlanması sağlanabilir:

```
// BASİT MUTEX ÇÖZÜMÜ
void philosopher(int i) {
    while (TRUE) {
        think();
        down(&mutex);          // Global kilidi al
        take_fork(i);
        take_fork((i+1) % N);
        up(&mutex);            // Kilidi bırak — Henüz YEMEDEN!
        eat();
        put_fork(i);
        put_fork((i+1) % N);
    }
}
```

Mutex Çözümü — Yalnızca 1 Filozof Aynı Anda Yiyebilir



Şekil 3: Mutex çözümü — yalnızca A yiyor, diğerleri bekliyor. Concurrency kaybı var.

⚠️ Mutex Çözümünün Sorunu

Mutex yemekten önce bırakılırsa: E mutex'i bırakır, A ve diğerleri aynı anda 1 çatal almaya başlayabilir — aynı deadlock sorunu yeniden ortaya çıkar. Mutex yemek sırasında tutulursa: aynı anda yalnızca 1 filozof yiyebilir — concurrency tamamen kaybolur, sistemin performansı 1/5 oranına düşer.

2.8 Doğru ve Adil Çözüm

Doğru çözüm şu prensiplere dayanır: Her filozofun durumu THINKING, HUNGRY veya EATING olarak izlenir. Çatal alma ve durumu değiştirme işlemleri kritik bölge içinde atomik yapılır. Her iki çatal da müsaitse yemeğe başlanır, değilse proses HUNGRY olarak işaretlenir ve bir semaphore üzerinde bloke olur. Çatallarını bırakan bir filozof, komşularının HUNGRY olup olmadığını kontrol eder; açlarsa uyandırır.

✓ Çözümün Doğruluğu — İspat Taslağı

E ve C aynı anda yiyor olsun. B ve D engellenmiştir. E bitince A ve D uyandırılır; D hâlâ engellenebilir çünkü C'nin çatallarına ihtiyacı vardır, ama A çalışabilir. C bitince B çalışabilir. İlerleme her durumda garantidir — deadlock ve starvation yok. Varsayım: yemek yeme sonlu bir aktivitedir ve her semaphore'da bir kuyruk vardır.

2.9 Readers and Writers Problemi

Ortak bir veritabanına eşzamanlı erişimi modelleyen bu problemde iki tür işlem söz konusudur: okuyucular (readers) ve yazıcılar (writers). Kural basittir: birden fazla okuyucu aynı anda okuyabilir; ancak yazıcı çalışırken ne okuma ne de başka yazma yapılabilir.

Durum	İzin Durumu	Neden?
Birden fazla reader aynı anda	İzinli ✓	Okuma veriyi değiştirmez, çakışma olmaz
Reader + Writer aynı anda	Yasak ✗	Yazma sırasında okuma tutarsız veri görebilir
Birden fazla Writer aynı anda	Yasak ✗	Eşzamanlı yazma veriyi bozar

Çözüm özeti: Bir sayaç (mutex ile korunan) aktif reader sayısını tutar. İlk reader gelen veritabanı mutex'ini kilitler. Sonraki readerlar sadece sayacı artırır. Son reader veritabanı mutex'ini serbest bırakır. Writer, veritabanı mutex'ini kilitleyerek tüm readerların bitmesini bekler.

⚠ Starvation Tehlikesi

Readerlar sürekli geliyorsa writer hiçbir zaman sıraya giremez — writer starvation. Çözüm: bekleyen writer varsa yeni readerlar bloke edilir. Ama bu sefer reader starvation riski doğar. Hangi tarafın öncelikli olacağı tasarım kararıdır.

2.10 Dersler ve Genel İlkeler

İlke	Açıklama
Basit mutex yetmez	Karmaşık senkronizasyon sorunları için genellikle birden fazla kilitleme mekanizması gerekir
Karmaşık algoritmalar tehlikelidir	Yazması ve anlaması zor — doğruluğunu ispatlayabilmek kritiktir
Adillik tanımlanmalı	Starvation ve indefinite overtaking gerçek sorunlardır; ne kadar tolerans gösterilecek belirlenmeli
Deadlock'u izle	Her senkronizasyon çözümü potansiyel deadlock kaynakları açısından gözden geçirilmeli

3. Deadlock (Kilitlenme)

Tanım · Koşullar · Modelleme · Genel Stratejiler

3.1 Deadlock Nedir? — Alice ve Bob Senaryosu

Deadlock, iki ya da daha fazla prosesin birbirinin tuttuğu kaynakları bekleyerek sonsuza dek bloke kaldığı durumdur. Klasik senaryo: Alice yazıcıyı almış ve tarayıcıyı istemektedir. Bob ise tarayıcıyı almış ve yazıcıyı beklemektedir. Her ikisi de diğerini beklediğinden hiçbiri ilerleyemez.

⚠ Deadlock

Alice yazıcıyı tutar, tarayıcıyı bekler. Bob tarayıcıyı tutar, yazıcıyı bekler. Hiçbiri ilerleyemez — bu bir deadlock'tur.

3.2 Tarihçe — OS Sorunundan Uygulama Sorununa

Deadlock sorunu işletim sistemlerinin ilk dönemlerinde yoğunlukla araştırılmış ve özellikle teorisyenlerin ilgisini çekmiştir. Ancak günümüzde deadlock artık bir OS sorunundan çok uygulama düzeyinde bir sorun olarak değerlendirilmekte; özellikle veritabanı sistemlerinde kritik önem taşımaktadır. Linux çekirdeği, uygulama prosesleri için herhangi bir deadlock tespiti ya da önlemesi yapmaz; yalnızca çekirdeğin kendi iç kilitleme mekanizmalarını yönetir.

3.3 Kaynaklar: Fiziksel, Soyut, Preemptable

Deadlock analizinde kaynak (resource), proseslerin talep ettiği ve tek seferde yalnızca bir prosese atanabilen her şeydir. Kaynaklar birçok farklı biçimde olabilir:

Kaynak Türü	Örnekler	Preemptable mı?
Fiziksel aygıt	Yazıcı, tarayıcı, teyp sürücüsü, CD-ROM	Hayır (iş ortasında alınamaz)
Veri kaydı	Veritabanı satırı, dosya kilit	Bağlama göre
Ana bellek bölgesi	Bellek segmenti, paylaşımlı bellek	Evet (swap varsa)
Soyut kaynak	Semaphore, mutex, mesaj kuyruğu, soket bağlantısı	Hayır

i Preemptable Kaynaklar ve Deadlock

Kaynaklar preemptable ise — yani bir prosesten zorla alınıp başkasına verilebiliyorsa — deadlock oluşamaz. Örneğin ana bellek swap desteğiyle preemptable'dır: bir prosesin bellek sayfaları diske taşınabilir ve kaynak başkasına verilebilir. Deadlock sorunu özünde non-preemptable kaynaklar etrafında döner.

3.4 Kaynak Kullanım Dizisi

Herhangi bir kaynağın kullanım sırası üç adımdan oluşur: önce kaynak talep edilir (request), kaynak müsaitse atanır, müsait değilse proses bloke olur ya da döngüsel olarak yeniden dener. Sonra kaynak kullanılır (use). Son olarak kaynak serbest bırakılır (release). Deadlock tehlikesi, bir prosesin birden fazla kaynak gerektirdiği durumda ortaya çıkar:

```
// DOĞRU SIRALAMA – her iki proses de önce resource_1 alır
process_A() {
    down(&resource_1);
    down(&resource_2);
    use_resources();
    up(&resource_1);
    up(&resource_2);
}
process_B() {
    down(&resource_1);
    down(&resource_2);
    use_resources();
    up(&resource_1);
    up(&resource_2);
}

// HATALI SIRALAMA – farklı sıra → deadlock riski!
process_A() {
    down(&resource_1);
    down(&resource_2);
    ...
}
process_B() {
    down(&resource_2); // ← farklı!
    down(&resource_1); // ← farklı!
    ...
}
```

⚠ Neden Farklı Sıra Deadlock Yaratır?

A resource_1'i alır ve resource_2'yi bekler. B aynı anda resource_2'yi almış ve resource_1'i beklemektedir. Circular wait oluşur — deadlock.

3.5 Coffman Koşulları (1971) — 4 Gerekli Koşul

Coffman, Elphick ve Shoshani 1971'de deadlock'un oluşabilmesi için dört koşulun eş zamanlı sağlanması gerektiğini kanıtlamıştır. Bu koşulların herhangi biri sağlanmıyorsa deadlock oluşamaz.

#	Koşul Adı	Tanım	OS Politikasıyla İlişkisi
1	Mutual Exclusion	Her kaynak ya tam olarak bir prosese atanmıştır ya da boştur	Kaynakların tek kullanımlı yapısından kaynaklanır
2	Hold and Wait	Kaynak tutan bir proses daha fazla kaynak talep edebilir	Kaynak talep protokolüne bağlıdır
3	No Preemption	Kaynaklar bir procesten zorla alınamaz; yalnızca sahip proses bırakabilir	Kaynak geri alma politikasına bağlıdır
4	Circular Wait	$P1 \rightarrow R1 \rightarrow P2 \rightarrow R2 \rightarrow \dots \rightarrow Pn \rightarrow Rn \rightarrow P1$ şeklinde döngüsel bekleme zinciri mevcuttur	Kaynak talep sırası politikasına bağlıdır

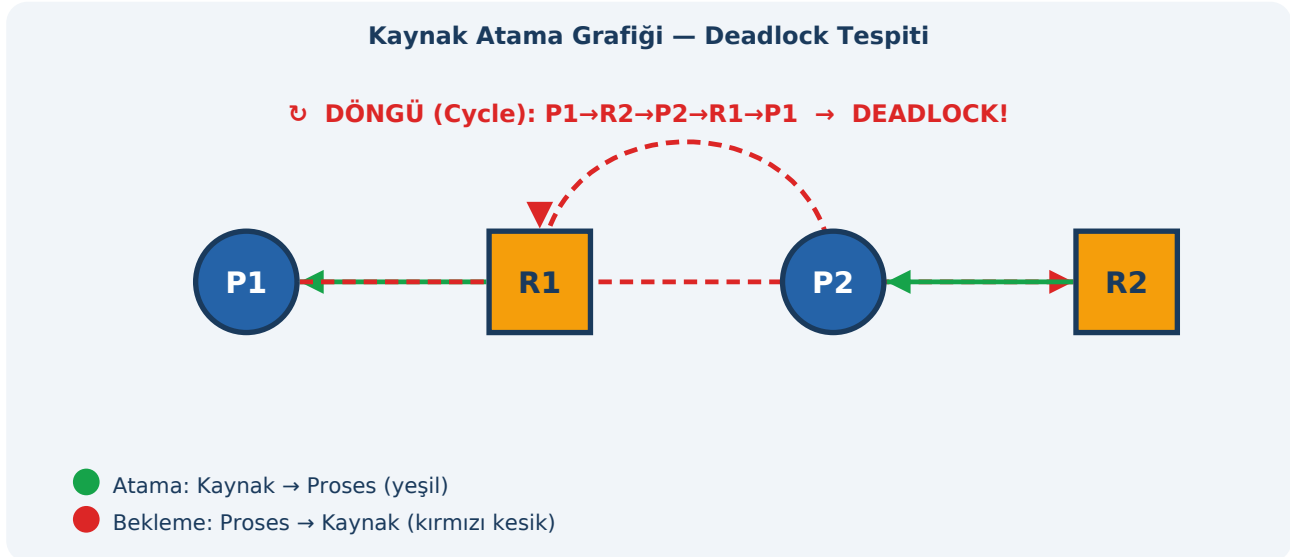
⚠ 4 Koşulun Tamamı Gereklidir

Bu koşullar AND ilişkisindedir — hepsi aynı anda sağlanmalıdır. Önemli not: bu koşulların hepsi doğrudan işletim sistemi politikasıyla bağlantılıdır. Deadlock önleme (prevention) stratejisi bu koşullardan en az birini ortadan kaldırmayı hedefler.

3.6 Kaynak Atama Grafiği ile Modelleme

Deadlock durumlarını görselleştirmek ve tespit etmek için yönlü graflar kullanılır. Graf iki tür düğüm içerir: proses düğümleri (daire ile gösterilir) ve kaynak düğümleri (kare ile gösterilir). Okların yönü ilişki türünü belirler:

Ok Yönü	Anlamı	Renk Kodu
Kaynak → Proses	Kaynak bu prosese atanmış (allocated)	Yeşil, düz ok
Proses → Kaynak	Proses bu kaynağı bekliyor (waiting)	Kırmızı, kesik ok



Şekil 4: Kaynak atama grafiği örneği. P1-R2-P2-R1-P1 döngüsü deadlock'a işaret eder. Her kaynak türünden yalnızca 1 kopya olduğunda döngü kesinlikle deadlock demektir.

Grafta bir döngü (cycle) bulunması deadlock varlığının göstergesidir. Her kaynak türünden yalnızca birer örnek mevcutsa, döngü bulunması deadlock olduğunu kesinlikle kanıtlar. Birden fazla kopya varsa döngü gerekli ama yeterli koşul değildir — ek analiz gerekmektedir.

3.7 Deadlock'tan Kaçınma — Graf Analizi

İşletim sistemi, bir kaynak talebi geldiğinde bu talebi onaylamadan önce yeni bir döngü oluşup oluşmayacağını kontrol edebilir. Döngü oluşacaksa talep geciktirilir; bunun yerine başka bir proses çalıştırılır. Bu yaklaşım, bazı kaynakların serbest bırakılmasını beklemeyi içerir.

3.8 Genel Stratejiler

Strateji	Ana Fikir	Maliyet	Pratik Kullanım
Görmezden gel	Sorun nadir; müdahale maliyeti fazla	Düşük	Linux, Windows — genel OS
Tespit & Kurtarma	Oluşmasına izin ver, sonra düzelt	Orta	Veritabanı sistemleri
Kaçınma	Güvenli durumda kalmayı garanti et	Yüksek	Teorik; pratikte nadir
Önleme	4 koşuldan birini imkânsız kıl	Çeşitli	Özel sistemler, mainframe

4. Deadlock ile Başa Çıkma — 4 Strateji

Görmezden gelme · Tespit & Kurtarma · Kaçınma · Önleme

4.1 Görmezden Gelme — Ostrich Algoritması

Eğer deadlock olasılığı düşük ve önleme maliyeti yüksekse, en pratik yaklaşım sorunu görmezden gelmektir; tıpkı bir devekuşunun başını kuma gömmesi gibi. Örneğin bir Linux sisteminde 532 proses ve 1772 açık dosya olabilir; teorik olarak deadlock mümkündür ancak pratikte neredeyse hiç gerçekleşmez.

i Overprovisioning ile Risk Azaltma

Bazı kaynakları (proses slotları gibi) fazladan tahsis etmek, deadlock riskini pratikte önemsiz düzeye indirir. Bu yaklaşım modern genel amaçlı işletim sistemlerinde yaygındır. Ancak veritabanı gibi kritik uygulamalarda yeterli değildir.

4.2 Tespit ve Kurtarma — Tek Kaynak Türü

Her kaynak türünden yalnızca bir kopya mevcut olduğunda, tespit işlemi çok basittir: kaynak atama grafi oluşturulur ve döngü aranır. Döngü varsa deadlock vardır. Döngü tespiti için $O(n^2)$ karmaşıklığında klasik graf algoritmaları kullanılır.

4.3 Tespit ve Kurtarma — Çoklu Kaynak Türü

Birden fazla kaynak türünden birden fazla kopya olduğunda matris tabanlı bir algoritma kullanılır. Aşağıdaki vektörler ve matrisler tanımlanır:

Sembol	Boyut	Tanım
E (Existing)	$1 \times m$ (m: kaynak türü sayısı)	Her kaynak türünün sistemdeki toplam kopya sayısı
A (Available)	$1 \times m$	Her kaynak türünün şu an atanmamış, serbest kopya sayısı
C (Current)	$n \times m$ (n: proses sayısı)	$C[i][j]$: Proses i'nin şu an tuttuğu j türü kaynak sayısı
R (Request)	$n \times m$	$R[i][j]$: Proses i'nin daha talep ettiği j türü kaynak sayısı

Değişmez (Invariant): Her j kaynak türü için: $\sum C[i][j] + A[j] = E[j]$

Yani her kaynak türünün tüm kopyaları ya bir prosese atanmıştır ya da serbesttir. Bu toplamın her zaman sabit (toplam kaynak sayısına eşit) olması gerekir.

Tespit Algoritması:

Adı	İşlem	Amaç
1	Tüm prosesler "işaretlenmemiş" başlar. İşaretlenmiş proses tamamlanabilir demektir.	Başlangıç durumu

Adı m	İşlem	Amaç
2	$R[i] \leq A$ koşulunu sağlayan işaretlenmemiş bir P_i bul.	Kimin çalışabileceğini belirle
3	P_i bulunduysa: $A = A + C[i]$ yap (bu proses bitince kaynakları serbest kalır). P_i 'yi işaretle. Adım 2'ye dön.	Kaynakları geri ver ve devam et
4	Böyle proses kalmadıysa dur. İşaretlenmemiş prosesler deadlock içindedir.	Deadlock kümesini tespit et

4.4 Deadlock Örneği Adım Adım

Sistem kaynakları: 4 Teyp Sürücüsü (TD), 2 Çizici (P), 3 Tarayıcı (S), 1 CD-ROM (CD).

	E (Toplam)	A (Serbest)		
	(4, 2, 3, 1)	(2, 1, 0, 0)		

Proses	C[TD]	C[P]	C[S]	C[CD]	R[TD]	R[P]	R[S]	R[CD]
P1	0	0	1	0	2	0	0	1
P2	2	0	0	1	1	0	1	0
P3	0	1	2	0	2	1	0	0

Algoritmanın Çalışması:

$A = (2,1,0,0)$. P1: $R1=(2,0,0,1) \leq (2,1,0,0)$? CD gerekiyor ama $A[CD]=0 \rightarrow$ Hayır. P2: $R2=(1,0,1,0) \leq (2,1,0,0)$? Tarayıcı gerekiyor ama $A[S]=0 \rightarrow$ Hayır. P3: $R3=(2,1,0,0) \leq (2,1,0,0)$? Evet! P3 işaretlenir, $A=(2,1,0,0)+(0,1,2,0)=(2,2,2,0)$. P2: $R2=(1,0,1,0) \leq (2,2,2,0)$? Evet! P2 işaretlenir, $A=(2,2,2,0)+(2,0,0,1)=(4,2,2,1)$. P1: $R1=(2,0,0,1) \leq (4,2,2,1)$? Evet! P1 işaretlenir. Tüm prosesler işaretlendi — DEADLOCK YOK.

✓ Sezgi

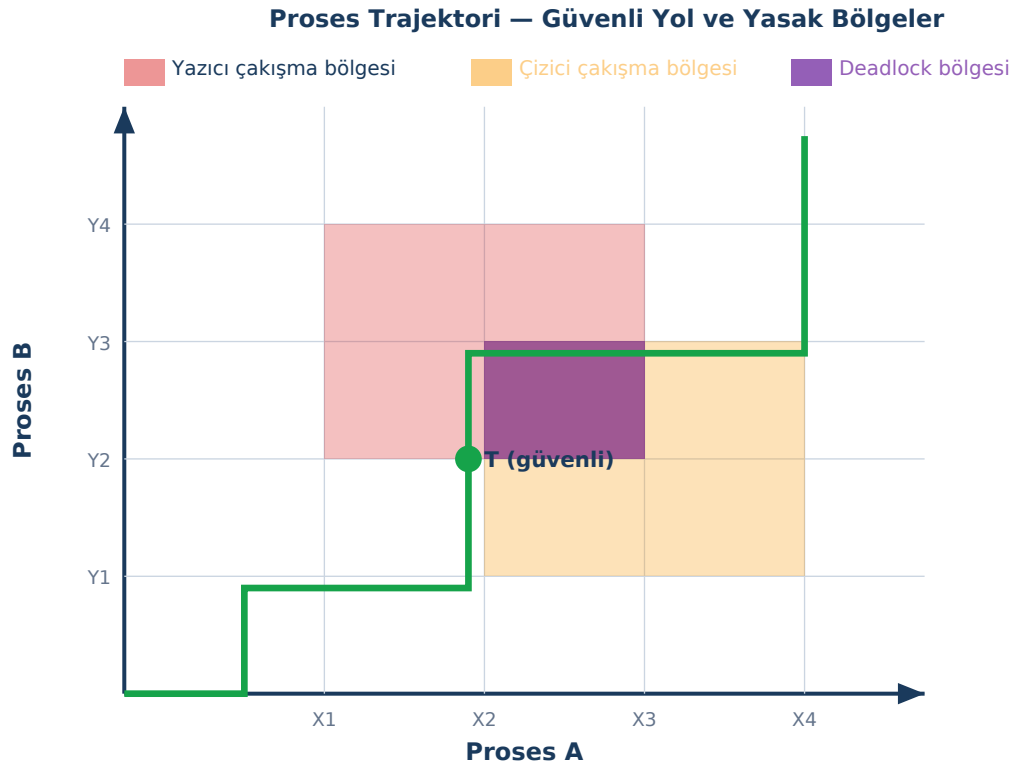
Algoritma her proses için şunu sorar: "Bu prosesin kalan talebini şu an karşılayabilir miyim? Evet ise onu çalıştırırım, kaynakları geri alırım ve devam ederim." Bu optimistik yaklaşım doğrudur: çalışabilen bir proses sonunda kaynakları iade eder.

4.5 Kurtarma Yöntemleri

Yöntem	Nasıl Çalışır?	Dezavantaj
Kaynak Preemption	Hangi kaynağın geri alınabileceği belirlenir, sahip proses geciktirilir	Her kaynak türü için mümkün olmayabilir; yazıcı, teyp gibi aygıtlar zordur
Geri Sarma (Rollback)	Prosesler periyodik checkpoint kaydeder; deadlock'ta uygun noktaya geri sarılır	Checkpoint maliyeti; veri tutarlılığının garanti edilmesi gerekir
Proses Öldürme	Döngüdeki (veya dışındaki) bir proses sonlandırılır; yan etkisi az olanı seç	Hangi prosesin öldürüleceği kritik karar; kısmi iş kaybı

4.6 Deadlock Kaçınma — Proses Trajektori Analizi

İki proses ve iki kaynak için çalışma uzayı 2B düzlemde çizilebilir. X eksenini Proses A'nın ilerleme durumunu, Y eksenini Proses B'ninki temsil eder. Aşağıdaki örnek bir yazıcı ve bir çizici için çakışma bölgelerini göstermektedir:

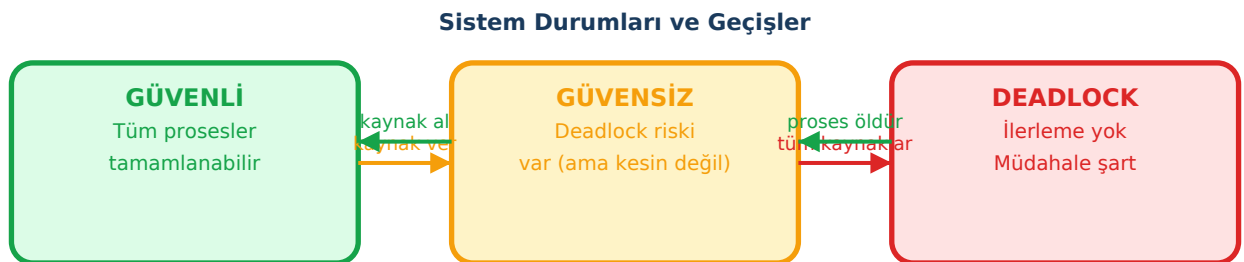


Şekil 5: Proses trajektori diyagramı. Renkli bölgeler iki prosesin aynı anda aynı kaynağı tutacağı imkânsız durumları gösterir. Mor kesişim bölgesi kesin deadlock bölgesidir. Yeşil çizgi güvenli trajektoriyi gösterir.

i T Noktasının Anlamı

T noktasında B çalıştırılırsa sistem yasak bölgeye girer ve deadlock oluşur. Bu nedenle scheduler'ın T'de A'yı çalıştırması gerekir. B, A'nın X3 noktasını geçmesini bekleyerek güvenli bölgeye girebilir.

4.7 Güvenli ve Güvensiz Durumlar



Şekil 6: Sistem durum geçişleri. Güvenli→Güvensiz geçişi olabilir; Güvensiz→Deadlock kesin değil ama risk yüksektir.

Güvenli durum: Sistem deadlock'ta değil ve mevcut kaynaklar ile beklenen serbest bırakmalar göz önüne alındığında, tüm proseslerin maksimum taleplerini karşılayarak tamamlanabileceği en az bir zamanlama sırası mevcuttur.

Güvensiz durum: Böyle bir sıralama mevcut değildir. Sistem şu an deadlock'ta olmayabilir; ama prosesler kötü zamanlama ile kaynak talep ederlerse deadlock kaçınılmaz olur. Güvensiz durumdan kurtulmak için proses kaynak talebi reddedilebilir ya da bir proses geciktirilebilir.

4.8 Banker's Algoritması (Dijkstra, 1965)

Bir bankanın müşterilerine kredi vermesine benzetilir: banka sınırlı miktarda nakit tutar; her müşterinin maksimum kredi limiti önceden bilinir; banka yalnızca güvenli durumda kalacağı talepleri onaylar.

Banker's Algoritması — Tek Kaynak Örneği (Serbest=2)

Proses	Sahip	Maksimum	Hâlâ Lazım
A	1	6	5
B	1	5	4
C	2	4	2
D	4	7	3

Serbest (Free) = 2

Güvenli sıra: C → A veya D → B → ...

Şekil 7: Banker's Algoritması örneği. Serbest=2 ile C'nin talebi karşılanır (hâlâ lazım=2). C tamamlayınca serbest=4, ardından A veya D çalışabilir. Sistem güvenli durumda.

Banker's Algoritması Örnek Tablosu:

Durum	Has	Max	Free	Güvenli mi?	Güvenli Sıra
Başlangıç	A=0,B=0,C=0,D=0	A=6,B=5,C=4,D=7	10	Evet	Herhangi sıra
Orta durum	A=1,B=1,C=2,D=4	A=6,B=5,C=4,D=7	2	Evet	C→A(ya da D)→...
B'ye 1 daha	A=1,B=2,C=2,D=4	A=6,B=5,C=4,D=7	1	Hayır	—

⚠ Banker's Neden Kullanılmıyor?

Her proses başlarken maksimum kaynak talebini bildirmelidir — modern sistemlerde neredeyse imkânsız. Prosesler sürekli gelip gider. Donanım arızalanabilir. Dinamik kütüphaneler, sistem çağrıları, runtime koşulları taleplerini belirsiz kılar. Sonuç: teorik olarak zarif, pratikte uygulanamaz.

4.9 Banker's Çoklu Kaynak İçin

Tek kaynak yerine m kaynak türü olduğunda aynı matris yapısı kullanılır (bkz. Tespit Algoritması). Güvenlilik testi şöyle çalışır:

Adı m	İşlem
1	C ve R matrislerini, E ve A vektörlerini oluştur.
2	İşaretlenmemiş P _i bul: $R[i] \leq A$ (tüm kaynak türleri için).
3	Bulunursa: $A = A + C[i]$. P _i 'yi işaretle. Adım 2'ye dön.
4	Tüm prosesler işaretlendiyse GÜVENLİ; işaretlenmemiş varsa GÜVENSİZ — talebi reddet.

4.10 Deadlock Önleme — 4 Koşula Saldırı

En köklü yaklaşım: Coffman koşullarından birinin sistemde hiç oluşmamasını garanti etmek.

Saldırılan Koşul	Yöntem	Nasıl Çalışır?	Sınırlaması
Mutual Exclusion	Spooling	Yazıcılar spool edilir; yalnızca printer daemon kaynağa erişir, yarışma olmaz	Tüm kaynaklar spool edilemez; birçok kaynak özünde exclusive'dir
Hold and Wait	Önceden talep	Proses başlamadan tüm ihtiyaç duyacağı kaynakları ister; yoksa bekler	Kaynaklar uzun süre boş bekler; kaynağın ne kadar gerektiği bilinmeyebilir
No Preemption	Serbest bırak ve yeniden iste	Yeni kaynak alınamıyorsa mevcut kaynaklar bırakılır, hepsi birden yeniden istenir	Bazı kaynaklar yarıda bırakılamaz (yazıcı, teyp)
Circular Wait	Kaynak numaralandırma	Tüm kaynaklara global sıra numarası atanır; prosesler yalnızca artan sırayla talep edebilir	Numaralandırma önceden yapılmalı; bazen uygunsuz sıralama zorunlu olabilir

Kaynak Numaralandırma — Neden Circular Wait'i Önler?

Tüm kaynak türlerine 1'den n'ye sıra numarası atanır. Proses i, kaynak j'yi almadan önce j'den küçük numaralı tüm kaynakları bırakmak zorundadır. Döngüsel bekleme için $P1 \rightarrow R_a \rightarrow P2 \rightarrow R_b \rightarrow P1$ gibi bir zincir gerekir; bu, $R_a < R_b < \dots < R_a$ demektir — matematiksel çelişki, imkânsızdır.

```
// Kaynak sırası: scanner=1, printer=2, tape=3, CD=4
// DOĞRU – artan sıra:
process_X() {
    down(&scanner);    // 1
    down(&printer);    // 2 (1 < 2 – geçerli)
    use_both();
    up(&scanner); up(&printer);
}
// HATALI – azalan sıra, circular wait mümkün:
process_Y() {
    down(&printer);    // 2
```

```
down(&scanner);    // 1  (2 > 1 – YASAK!)
}
```

4.11 İki Aşamalı Kilitleme (Two-Phase Locking)

Veritabanı sistemlerinde sıkça kullanılan bu teknik, deadlock önlemenin dinamik bir versiyonudur. Proses birden fazla kaydı kilitlemeden güncelleyemez.

Faz	İşlem	Başarısız Olursa
Faz 1 — Kilitleme	Kayıtlar birer birer kilitlenmeye çalışılır	Bir kayıt kilitliyse tüm kilitler bırakılır ve baştan başlanır
Faz 2 — Güncelleme & Bırakma	Tüm kilitler alındıktan sonra güncellemeler yapılır ve kilitler serbest bırakılır	—

i Bağlantı: "Her Şeyi Önceden Talep Et" ile İlişki

Two-Phase Locking, "tüm kaynakları önceden talep et ve yoksa bekle" yaklaşımının dinamik bir uygulamasıdır. Her başarısız girişimde tüm kilitler serbest bırakıldığından hold-and-wait koşulu bozulur.

4.12 Linux'ta Deadlock ve Fork Bomb

Linux çekirdeği uygulama prosesleri için herhangi bir deadlock önleme ya da tespiti yapmaz. Kernel, kendi iç kilitleme mekanizmaları için dikkatli tasarım uygular. Fork bomb ise deadlock'tan farklı ama ilgili bir kaynak tükenmesi saldırısıdır:

```
# Fork Bomb – Python (ÇALIŞTIRMAYIN!)
import os
while True:
    os.fork()    # Proses sürekli kendini kopyalar

# Bash versiyonu:
# :(){ :|:& };:

# Açıklama: her proses 2 çocuk üretir → 2^n proses
# Proses tablosu dolunca sistem cevap veremez hale gelir (DoS)
```

⚠ Fork Bomb Koruması

Linux'ta "ulimit -u " ile kullanıcı başına maksimum proses sayısı kısıtlanabilir. Bu fork bomb'u tamamen engellemez ama sistemi korur.

i Linux Kernel'inin Tutumu

"İşlevsel olmayan bir socketi doldurup deadlock'a neden olabileceğimiz için tüm kuralları burada döküp çıkış yapamayız..." (Linux kaynak kodu yorumu). Kernel kendi iç deadlock'larını son derece dikkatli tasarımla yönetir — uygulama katmanı için bu sorumluluğu geliştiriciyle paylaşır.

5. Sınav Soruları ve Çözümleri

Soru 1: Linux çekirdeğinde jiffies değişkeninin neden 64 bit olarak tanımlandığını açıklayın. 32-bit sürümün yarattığı sorunu ve çözümü (spin-lock + linker magic) detaylıca anlatın.

Cevap: 1000 Hz'de çalışan 32-bit jiffies yaklaşık 49.7 günde taşar (wrap-around). Uzun çalışan sunucularda bu durum, zaman karşılaştırmalarında yanlış sonuçlar doğurur. Çözüm olarak 64-bit jiffies_64 tanımlanır. Ancak 32-bit makinede 64-bit değere atomik erişim imkânsızdır: iki 32-bit okuma gerekmekte, bu okumalar arasında bir kesme gelirse tutarsız değer elde edilmektedir (race condition). Çözüm: (1) jiffies_64'e erişim get_jiffies_64() fonksiyonu aracılığıyla spin-lock koruması altında yapılır. (2) Linker magic: "jiffies" sembolü jiffies_64'ün düşük 32 bitine yönlendirilir; her kesmede yalnızca jiffies_64 bir kez artırılır, hem "jiffies" hem de "jiffies_64" otomatik güncel kalır.

Soru 2: Dining Philosophers problemindeki naif çözümün neden deadlock'a yol açtığını gösterin. Coffman koşullarını bu senaryoya uygulayın.

Cevap: Naif çözümde her filozof sırayla sol, sonra sağ çatalı alır. Tüm filozoflar eş zamanlı olarak sol çatalarını alırsa — preemptive sistemde bu son derece mümkündür — her biri sağdaki çatalı bekler. Sağ çatal ise komşunun sol çatalıdır, o da tutulmuş. Coffman koşulları: (1) Mutual Exclusion — çatal aynı anda yalnızca bir filozofa ait. (2) Hold and Wait — sol çatalı tutan filozof sağ çatalı bekler. (3) No Preemption — çatal zorla alınamaz. (4) Circular Wait — A sol çatalını tutar ve B'nin bırakmasını bekler; B sol çatalını tutar ve C'nin bırakmasını bekler; ... ; E A'yı bekler. Döngüsel zincir oluşur. Dört koşulun tümü sağlanmaktadır — deadlock kaçınılmaz.

Soru 3: Dining Philosophers için doğru ve adil çözümün temel prensiplerini açıklayın. Çözümün deadlock ve starvation'ı neden önlediğini ispat taslağıyla gösterin.

Cevap: Prensip: (1) Her filozofun durumu THINKING/HUNGRY/EATING olarak izlenir. (2) Çatal alma ve durum değiştirme kritik bölge içinde atomik yapılır. (3) Her iki komşu yemiyorsa yemek başlanır; aksi halde HUNGRY + semaphore.down(). (4) Çatalar bırakılırken komşuların HUNGRY olup olmadığı kontrol edilir, öyleyse uyandırılır. İspat taslağı: E ve C yiyor olsun. A ile D engelli. B ve D'nin de engelli olduğunu varsayalım. E bitince A ve D uyandırılır. D C'nin çatalını beklediğinden hâlâ engelli olabilir; ama A çalışabilir (her iki komşusu da yemiyor). C bitince B uyandırılır. İlerleme her durumda garantidir. Varsayım: yemek yeme sonludur ve her semaphore'da FIFO kuyruk vardır.

Soru 4: Aşağıdaki sisteme deadlock tespit algoritmasını adım adım uygulayın: $E=(3,3,2)$, $A=(0,0,0)$, $C=[(1,0,1),(0,2,0),(1,1,0)]$, $R=[(1,1,0),(0,0,2),(1,0,1)]$.

Cevap: Başlangıç: $A=(0,0,0)$. P1: $R_1=(1,1,0) \leq A=(0,0,0)$? $1>0 \rightarrow$ Hayır. P2: $R_2=(0,0,2) \leq A=(0,0,0)$? $2>0 \rightarrow$ Hayır. P3: $R_3=(1,0,1) \leq A=(0,0,0)$? $1>0 \rightarrow$ Hayır. Hiçbir proses çalışmıyor, algoritma durur. Tüm prosesler işaretlenmemiş $\rightarrow$ DEADLOCK: P1, P2 ve P3 kilitlenmiş. Kurtarma seçenekleri: P2'yi öldür (2 kaynak tutmuş, 2 kaynak istiyor — orta maliyetli); ardından $A=(0,2,0)$, P1 çalışabilir. P1 bitince $A=(1,2,1)$, P3 çalışabilir.

Soru 5: Güvenli ve güvensiz durum arasındaki farkı açıklayın. Bir durumun güvenli olduğunu nasıl test edersiniz? Güvensiz durumun her zaman deadlock demek olmadığını örnekle gösterin.

Cevap: Güvenli durum: sistem deadlock'ta değil VE mevcut kaynaklar göz önüne alındığında tüm proseslerin tamamlanabileceği en az bir zamanlama sırası var. Test: Banker's güvenlilik kontrolü — işaretlenmemiş proses $R[i] \leq A$ ise işaretler, $A = A + C[i]$, tekrarla; tüm işaretlendiyse güvenli. Güvensiz $\neq$ Deadlock: Güvensiz durumda sistem hâlâ çalışıyor olabilir. Eğer prosesler maksimum taleplerini kullanmadan kaynakları bırakırlarsa deadlock olmaz. Ancak en kötü durumda (tüm maksimum talepler kullanılırsa) deadlock kaçınılmaz. Banker's algoritması güvensiz duruma hiç girmemeyi garanti eder.

Soru 6: Deadlock'u önlemenin dört yolunu açıklayın. Her biri için gerçek dünya uygulamalarından bir örnek verin ve pratikte ne kadar kullanılabilir olduğunu tartışın.

Cevap: (1) Mutual Exclusion'a Saldırı: Yazıcıları spool etmek — spool dizinine herkes yazabilir, yalnızca printer daemon yazıcıya erişir. Pratikte sınırlı; tüm kaynaklar spool edilemez. (2) Hold and Wait'e Saldırı: Mainframe batch işlerde tüm dosyalar önceden bildirilir. Pratikte modern sistemlerde neredeyse imkânsız. (3) No Preemption'a Saldırı: Bellek sayfaları swap edilebilir — bellek preemptable. Yazıcı/teyp gibi aygıtlar için imkânsız. (4) Circular Wait'e Saldırı: Kaynak numaralandırma — eski mainframe'lerde sıkça kullanılmış. Scanner < Printer < Tape sırasıyla talep zorunluluğu. İki aşamalı kilitleme (veritabanı) pratikte yaygın. Genel not: Modern OS hiçbirini otomatik uygulamaz; sorumluluk geliştiriciye aittir.

Soru 7: Bir veritabanı sistemi deadlock tespiti çalıştırıyor ve deadlock bulundu. Üç farklı kurtarma stratejisini karşılaştırın: hangi durumda hangisini tercih edersiniz?

Cevap: (1) Kaynak Preemption: Bir procesten kaynak zorla alınır, başkasına verilir, sonra geri iade edilir. En uygun durum: kaynak preemptable ise (bellek gibi). Hangi proses geciktirileceği seçilmeli — en düşük maliyetli tercih edilir. (2) Rollback: Proses checkpoint'e geri sarılır ve kaynakları bırakır. En uygun durum: işlem yeniden çalıştırılabilirse ve veritabanı transaction semantiği destekliyorsa. ACID transaction'lar için ideal. (3) Proses Öldürme: En basit yöntem; döngüdeki bir proses kill edilir. Tercih kriteri: en az iş tamamlamış, en az yan etkili proses seçilmeli (kısmen yazılmış dosyalar, yarı güncellenmiş kayıtlar sorun çıkarır). Veritabanında transaction rollback mekanizması varsa rollback tercih edilir.

6. Kitap Sonu Soruları ve Çözümleri

Tanenbaum Modern Operating Systems 5th Ed. — Chapter 2 Problems

Problem 19 — Race Condition Tespiti

Soru: İki thread aynı anda get_account() fonksiyonunu çalıştırıyor. accounts[account_count] = entered_account ve account_count++ satırlarında race condition'ı belirtin.

Çözüm: Race condition accounts[account_count] = entered_account ve ardından account_count++ satırları arasında bulunmaktadır. Senaryo: Thread A account_count=5'i okur ve kesmeye uğrar. Thread B de account_count=5'i okur, accounts[5]=X yazar ve count'u 6 yapar. Thread A devam eder: accounts[5]=Y yazar (B'nin yazdığı X kaybolur!) ve count'u 7 yapar. Sonuç: account_count=7 ama yalnızca 1 kayıt geçerli; bir kez fazla artırıldı; B'nin girişi silindi. *Çözüm:* mutex ile accounts[account_count]=... ve account_count++ atomik yapılmalı.

Problem 21 — Kullanıcı Seviyesi Thread + Saat Kesmesi

Soru: Kullanıcı seviyesi thread sistemi saniyede 1 kez saat kesmesi alıyor. Saat kesmesi tam çalışma zamanı sisteminde bir thread bloke/debloke edilirken gelirse ne sorun oluşur? Çözebilir misiniz?

Çözüm: Sorun: Saat kesmesi, çalışma zamanı sistemi bir thread'i bloke veya uyandırırken tetiklenirse, çalışma zamanı sisteminin kendi veri yapıları (ready queue, blocked queue) tutarsız duruma girebilir. Örneğin thread henüz yarı blokluken kesme gelirse ve kesme başka bir thread çalıştırmaya çalışırsa, yarım durumda bir thread çalıştırılmış olur. *Çözüm yaklaşımları:* (1) Kritik bölgelerde kesmeleri geçici olarak devre dışı bırak (disable interrupts). (2) Çalışma zamanı sistemine girişte flag koy; saat kesmesi bu flag'i görürse asıl zamanlama kararını erteler, "bekleyen kesme" olarak işaretler.

Problem 23 — Peterson Çözümü ve Preemption

Soru: Peterson'ın mutual exclusion çözümü preemptive scheduling ile çalışır mı? Nonpreemptive ile?

Çözüm: Preemptive scheduling ile Peterson'ın çözümü ÇALIŞIR. Temel akıl yürütme: Peterson algoritması shared memory üzerinde atomik okuma/yazma varsayımıyla çalışır. Preemption sadece işlem sırasını değiştirir; algoritmanın doğruluğu tüm olası çalışma sıralarında geçerlidir (mutual exclusion, liveness ve bounded waiting hepsi sağlanmış). Nonpreemptive ile de ÇALIŞIR: proses CPU'yu bırakmadan çalıştığında zaten başka proses çalışamaz; bu daha az zorlayıcı bir durum. Not: çok işlemcili sistemlerde (SMP) Peterson'ın çözümü ek memory barrier'lar gerektirebilir.

Problem 25 — Priority Inversion + Round-Robin

Soru: Priority inversion sorunu (H yüksek, L düşük öncelik) round-robin scheduling ile de oluşur mu?

Çözüm: Saf round-robin ile priority inversion genellikle oluşmaz çünkü tüm prosesler aynı önceliğe sahip ve sırayla çalışır; M bir proses L'yi uzun süre bloke edemez. Ancak şu durumlarda sorun yeniden ortaya çıkabilir: (1) Öncelik sınıfları içinde round-robin: eğer M, L'den yüksek sınıftaysa L çalışamayabilir. (2) Yoğun CPU kullanan prosesler: M CPU'yu aktif kullanıyorsa L'nin quantum'u gecikmeli gelir, lock tutma süresi uzar, H bloke kalır. Genel ders: priority inversion lock semantiği + öncelik/scheduling kombinasyonundan kaynaklanır; yalnızca priority scheduling'e özgü değildir.

Problem 27 — Race Condition Tanımı

Soru: Race condition nedir? Formal bir tanım verin.

Çözüm: Race condition: iki veya daha fazla proses/thread'in paylaşılan bir kaynağa (değişken, dosya, I/O aygıtı) eş zamanlı eriştiği ve programın sonucunun bu erişimlerin görelili zamanlamasına (execution order) bağlı olduğu durumdur. Başka bir deyişle: sonuç deterministik değildir; farklı çalışmalarda farklı sonuçlar üretebilir. Kritik bölge (critical section) koruması olmaksızın yapılan her read-modify-write işlemi potansiyel bir race condition kaynağıdır. Örnek: x++ işlemi 3 assembly talimatından oluşur (LOAD, ADD, STORE). İki thread aynı anda x++ işlemine girerse, her ikisi de aynı eski değeri okuyabilir, her biri x'i 1 artırır ve yazar — net sonuç: x yalnızca 1 artar, 2 değil.

Problem 37 — Round-Robin Listede Çift Giriş

Soru: Round-robin kuyruğunda bir proses iki kez yer alırsa ne olur? Buna neden izin vermek isteyebilirsiniz?

Çözüm: Etki: Her turda iki kez quantum alır — normal proses CPU zamanının 2 katını tüketir. Gerekçe: Bu basit ama etkili bir öncelik mekanizmasıdır. k kez listelenen proses, 1 kez listelenenin k katı CPU zamanı alır. Lottery scheduling'in deterministik versiyonu olarak düşünülebilir. Kullanım alanı: kritik arka plan proseslerini (daemon) hızlandırmak. Dezavantajlar: (1) Proses sonlanınca veya bloke olunca tüm kopyalarının listeden çıkarılması gerekir; bu hata yapılırsa "hayalet proses girişleri" kalır. (2) Liste yönetimi karmaşılaşır. (3) Gerçek öncelik sistemleri bu amaca daha iyi hizmet eder.

Problem 53 — Readers-Writers Varyasyonları

Soru: Readers-Writers probleminin reader veya writer'ı farklı biçimde önceliklendiren üç farklı versiyonunu tanımlayın.

Çözüm: (1) Reader Öncelikli (Readers' Preference): Bekleyen reader varsa yeni reader hemen girer, mevcut writer'lar bitmeden yeni okuma başlayabilir. Writer sadece hiç aktif reader kalmadığında çalışır. Tehlike: writer starvation — sürekli reader gelirse writer hiç çalışamaz. (2) Writer Öncelikli (Writers' Preference): Writer bekliyorsa yeni readerlar bloke edilir. Mevcut readerlar biter bitmez writer girer. Reader sadece hiç bekleyen writer yoksa girebilir. Tehlike: reader starvation — sürekli writer gelirse okuma durur. (3) Adil (Fair / FIFO): Hem readerlar hem writerlar geliş sırasına göre kuyruğa alınır. Aynı anda gelen birden fazla reader gruplanabilir (birlikte okurlar). Ne reader ne writer süresiz beklemez. Tercih: çoğu gerçek sistemde 3. varyasyon tercih edilir; veritabanlarında genellikle writer öncelikli kullanılır.
